

高级语言程序设计 实验报告

南开大学 密码与网络安全学院

姓名：张藐

学号：2310558

班级：密码科学与技术

目录

高级语言程序设计大作业实验报告	3
1. 作业题目	3
2. 开发软件	3
3. 课题要求	3
4. 主要流程	3
4.1 整体流程 / 设计思路	3
4.2 算法或公式	4
4.3 模块 / 功能说明	5
4.4 关键代码解释	6
5. 单元测试	10
5.1 测试用例	11
5.2 开发过程遇到的问题与解决方法	11
6. 实验收获	13
附录:	14
A. 关键代码位置	14
B. AI 工具使用披露	14

高级语言程序设计大作业实验报告

1. 作业题目

本次大作业实现一个基于 L-System 的分形植物生成器，能够根据重写规则生成植物图形，并支持参数调节、动画生长、背景主题切换与音效播放。

一句话概括任务目标：把“字符串重写规则”转化为“可交互、可演示、可保存”的分形植物程序。

2. 开发软件

- 编程语言：C++。
- 图形界面框架：Qt 6 Widgets。
- 多媒体模块：Qt Multimedia（FFmpeg 后端）。
- 开发环境：Visual Studio 2022、Qt 6.10.2 MSVC2022、CMake。
- 运行平台：Windows 10/11。

3. 课题要求

1. 使用面向对象方法组织代码，合理划分类与函数。
2. 通过虚函数和虚析构函数实现多态调用和资源安全释放。
3. 完成基本图形生成与显示，并支持参数可调。
4. 提供单元测试或测试用例，能够验证输入与输出是否符合预期。
5. 在实现功能的同时保证代码结构清晰，便于扩展和维护。

本项目在满足核心功能的基础上，进一步扩展了动画、主题、音效和文件保存等增强功能，以提高作品完整度。

4. 主要流程

4.1 整体流程 / 设计思路

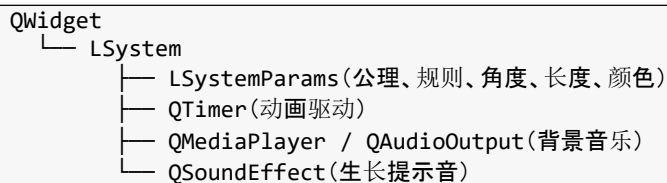
程序采用“窗口类 + 参数结构 + 绘制函数 + 音效模块”的结构。LSystem 继承自 QWidget，通过重写绘制事件、鼠标事件和尺寸变化事件完成界面显示与交互。

从面向对象角度看，程序把界面、算法和音效统一封装在 LSystem 类中，而不是把所有逻辑都写在 main 函数里。LSystem 继承自 QWidget，说明它既是一个窗口对象，也是一个可以参与 Qt 事件循环的控件对象；通过重写 paintEvent、resizeEvent

和 `mousePressEvent` 等虚函数，Qt 会在运行时把事件分发给派生类的实现，这就是课程中强调的运行时多态。

同时，`LSystemParams` 负责保存生成参数，`PlantPreset` 和 `BackgroundTheme` 用枚举表示可选状态，这样把“数据”与“行为”分开，降低了耦合度，也让后续新增植物预设或背景主题时只需要扩展枚举和规则，不必重写整套绘制流程。

核心类关系可以概括为：



类定义与重写接口的关键代码如下：

```
class LSystem : public QWidget
{
    Q_OBJECT

protected:
    void paintEvent(QPaintEvent* event) override;
    void resizeEvent(QResizeEvent* event) override;
    void mousePressEvent(QMouseEvent* event) override;
};
```

整体流程为：启动程序后初始化 UI 和音效系统，生成初始 L-System 字符串；用户调整参数后重新生成字符串并重绘；点击“动画生长”时由定时器逐步推进绘制；音效在生长阶段和背景播放阶段同步触发。

4.2 算法或公式

(1) 字符串重写规则是本项目的核心算法。设第 n 次迭代得到字符串 S_n ，则下一次迭代满足：

$S_{(n+1)} = R(S_n)$
其中 $R(\cdot)$ 表示按重写规则逐字符替换。
例如： $F \rightarrow FF+[F-F-F]-[-F+F+F]$

对应的生成代码如下：

```
QString LSystem::generateLSystemString()
{
    QString result = m_params.axiom;
    for (int i = 0; i < m_iterations; ++i) {
        QString newResult;
        for (const QChar& c : result) {
            if (m_params.rules.contains(c)) {
                newResult += m_params.rules[c];
            } else {
                newResult += c;
            }
        }
        result = newResult;
    }
}
```

```
    return result;
}
```

(2) 海龟绘图中, F 表示向前画线, + / - 表示角度旋转。若当前角度为 θ , 基础角度为 α , 扰动为 δ , 则:

```
 $\theta_{\text{new}} = \theta + \alpha + \delta$     (左转)
 $\theta_{\text{new}} = \theta - \alpha + \delta$     (右转)
 $\delta \in [-2^\circ, 2^\circ]$ , 用于增加自然随机性。
```

(3) 枝条粗细渐变用于表现植物层次感, 代码中使用的近似公式为:

```
thickness = max(1.0, 4.0 * (1.0 - ratio * 0.8))
ratio 表示当前绘制进度, 值越大, 枝条越细。
```

对应绘制核心片段如下:

```
if (c == 'F') {
    double rad = angle * PI / 180.0;
    double newX = x + m_params.length * cos(rad);
    double newY = y - m_params.length * sin(rad);
    double ratio = static_cast<double>(i) / drawCount;
    double thickness = 4.0 * (1.0 - ratio * 0.8);
    thickness = qMax(1.0, thickness);
    painter.drawLine(QPointF(x, y), QPointF(newX, newY));
}
```

4.3 模块 / 功能说明

函数 / 模块	作用
initUI()	创建参数面板、按钮、复选框和整体布局。
generateLSystemString()	根据公理和规则生成 L-System 字符串。
drawBackground()	绘制草地、星空和雪景三种背景。
drawLSystem()	按字符解释并逐步绘制植物。
updateAnimation()	推进动画步数并播放阶段性音效。
onPresetChanged()	切换不同植物预设并刷新字符串。
randomizeParams()	随机调整角度、长度和颜色, 让植物更自然。
initSoundSystem()	创建背景音乐播放器与音效对象。
resolveBackgroundMusicPath()	按可执行文件路径和资源路径查找背景音乐。
extractResourceToTempFile()	将 qrc 中的背景音乐导出到临时目录后播放。

音效系统的关键初始化代码如下:

```
void LSystem::initSoundSystem()
{
    m_backgroundPlayer = new QMediaPlayer(this);
    m_backgroundAudio = new QAudioOutput(this);
    m_backgroundPlayer->setAudioOutput(m_backgroundAudio);
    m_backgroundAudio->setVolume(0.5);
    m_backgroundPlayer->setLoops(QMediaPlayer::Infinite);
}
```

资源路径兼容处理的关键代码如下:

4.4 关键代码解释

4.4.1 类设计与继承关系

```
class LSystem : public QWidget
{
    Q_OBJECT

public:
    LSystem(QWidget* parent = nullptr);
    ~LSystem() override;

protected:
    void paintEvent(QPaintEvent* event) override;
    void resizeEvent(QResizeEvent* event) override;
    void mousePressEvent(QMouseEvent* event) override;

private:
    enum PlantPreset { ClassicPlant, Fern, Tree };
    enum BackgroundTheme { Grassland, StarrySky, SnowScene };

    struct LSystemParams {
        QString axiom;
        QMap<QChar, QString> rules;
        double angle;
        double length;
        QColor stemColor;
        QColor tipColor;
    };
};
```

这段代码体现了典型的面向对象设计：LSystem 作为核心窗口类直接继承 QWidget，复用了 Qt 的窗口、绘制和事件机制；Q_OBJECT 让类具备元对象能力，便于信号与槽工作；虚析构函数保证通过基类接口销毁对象时资源能够正确释放。

PlantPreset 和 BackgroundTheme 用枚举限定取值范围，LSystemParams 用结构体集中保存当前植物的生成参数。这样的类设计把“窗口控制”“算法参数”“界面交互”放在同一个对象中协同工作，逻辑清楚，也方便扩展。

4.4.2 字符串重写与海龟绘图

```
QString LSystem::generateLSystemString()
{
    QString result = m_params.axiom;

    for (int i = 0; i < m_iterations; ++i) {
        QString newResult;
        for (const QChar& c : result) {
            if (m_params.rules.contains(c)) {
                newResult += m_params.rules[c];
            } else {
                newResult += c;
            }
        }
        result = newResult;
    }

    return result;
}
```

该函数实现了 L-System 的字符串重写规则：先从公理字符串 axiom 出发，再按迭代次数不断用规则表替换字符。每一轮迭代都相当于对上一轮结果做一次“展开”，最终得到可用于绘图的植物指令串。

绘制部分使用海龟绘图思想：F 表示前进并画线，+/- 表示旋转，[] 用栈保存和恢复位置与角度。这样就能自然地表达分支结构；同时代码中加入随机角度扰动和颜色渐变，使植物形态更接近真实生长效果。

```
void LSystem::drawLSystem(QPainter& painter, int step)
{
    QStack<QPair<QPointF, double>> stack;

    int drawCount = (step == -1) ? m_ksystemString.length() : step;

    for (int i = 0; i < drawCount; ++i) {
        QChar c = m_ksystemString[i];

        if (c == "F") {
            double rad = angle * PI / 180.0;

            double newX = x + m_params.length * cos(rad);
            double newY = y - m_params.length * sin(rad);
            painter.drawLine(QPointF(x, y), QPointF(newX, newY));
            x = newX;
            y = newY;
        } else if (c == "+") {
            angle += m_params.angle + (std::rand() % 5 - 2);
        } else if (c == "-") {
            angle -= m_params.angle + (std::rand() % 5 - 2);
        } else if (c == "[") {
            stack.push(QPair<QPointF, double>(QPointF(x, y), angle));
        } else if (c == "]") {
            QPair<QPointF, double> state = stack.pop();
            x = state.first.x();
            y = state.first.y();
            angle = state.second;
        }
    }
}
```



```

    }
}

```

这里把字符串中的符号直接映射为几何操作，形成“规则生成字符串，字符串驱动绘图”的完整链路。由于分支状态通过栈保存，算法能够正确恢复到上一个分叉点继续生长，这也是分形植物能够层层分叉的原因。

4.4.3 音效与资源兼容

```

void LSystem::initSoundSystem()
{
    m_backgroundPlayer = new QMediaPlayer(this);
    m_backgroundAudio = new QAudioOutput(this);
    m_backgroundPlayer->setAudioOutput(m_backgroundAudio);
    m_backgroundAudio->setVolume(0.5);
    m_backgroundPlayer->setLoops(QMediaPlayer::Infinite);

    const QString backgroundFilePath = resolveBackgroundMusicPath();
    if (!backgroundFilePath.isEmpty()) {
        m_backgroundPlayer->setSource(QUrl::fromLocalFile(
backgroundFilePath));
    }

    configureToneEffect(m_growthSound, QStringLiteral
("lsystem_growth.wav"), 740.0, 120, 0.18);
}

```

这部分体现了音频模块的对象分工：QMediaPlayer 负责媒体播放，QAudioOutput 负责输出通道，QSoundEffect 负责短音效。播放器不直接硬编码资源路径，而是先调用路径解析函数，再统一把背景音乐转换成可以稳定播放的本地文件。

```

QString LSystem::resolveBackgroundMusicPath() const
{
    const QStringList candidates = {
        QDir(appDir).filePath(QStringLiteral("audio/background3.mp3")),

```

```

        QDir(appDir).filePath(QStringLiteral("../audio/background3.mp3")),
        QDir(appDir).filePath(QStringLiteral("audio/background.mp3")),
        QDir(appDir).filePath(QStringLiteral("../audio/background.mp3"))
    };

    QFile resourceFile(QStringLiteral(":/audio/background3.mp3"));
    if (!resourceFile.exists()) {
        resourceFile.setFileName(QStringLiteral(":/audio/audio/background3.mp3"));
    };
    if (resourceFile.exists()) {
        return extractResourceToTempFile(resourceFile.fileName(),
        QStringLiteral("lsystem_background3.mp3"));
    }
    return QString();
}

```

该函数解决的是课程项目里最实际的问题之一：资源文件在不同构建目录和 `qrc` 层级下可能出现路径不一致。这里先查找可执行文件附近的磁盘资源，再回退到 `qrc` 资源并导出到临时目录，从而保证背景音乐在调试和发布环境中都能稳定播放。

```

QString LSystem::resolveBackgroundMusicPath() const
{
    const QStringList candidates = {
        QDir(appDir).filePath("audio/background3.mp3"),
        QDir(appDir).filePath("../audio/background3.mp3"),
        QDir(appDir).filePath("audio/background.mp3"),
        QDir(appDir).filePath("../audio/background.mp3")
    };
    ...
}

```

5. 单元测试

测试目标是验证不同输入下的输出是否正确，以及在异常情况下程序是否能给出明确反馈。测试重点包括：植物预设切换、字符串生成、动画播放、背景音乐加载和保存功能。

5.1 测试用例

测试用例	输入	预期输出	测试目的
用例 1	选择“经典植物”，迭代 4 次	生成经典分形植物	验证默认规则和绘制逻辑
用例 2	切换为“蕨类”预设	字符串规则和画面立即变化	验证预设切换是否生效
用例 3	点击“动画生长”	逐步绘制并播放背景音乐	验证定时器与动画状态
用例 4	点击界面任意位置	植物原点移动并重新生成	验证鼠标事件与随机参数
用例 5	背景音乐加载失败场景	控制台提示资源错误并回退	验证音频容错策略

5.2 开发过程遇到的问题与解决方法

问题 1：QMediaPlayer 头文件找不到

- 问题描述：在项目中加入背景音乐功能后，包含 `#include <QMediaPlayer>` 时编译器报错：无法打开包括文件：“QMediaPlayer”: No such file or directory。
- 错误原因：Qt 模块依赖缺失：QMediaPlayer 属于 Qt Multimedia 模块，不属于默认的 Widgets 模块；CMake 配置不完整：项目的 CMakeLists.txt 中只引入了 Widgets，未声明并链接 Multimedia；
- 解决方法

① 在 CMakeLists.txt 中修改 `find_package`，添加 Multimedia：

```
cmake
find_package(Qt${QT_VERSION_MAJOR} REQUIRED COMPONENTS Widgets Multimedia)
```

② 在 `target_link_libraries` 中链接多媒体模块：

```
cmake
target_link_libraries(LSystemPlant PRIVATE
Qt${QT_VERSION_MAJOR}::Widgets
Qt${QT_VERSION_MAJOR}::Multimedia)
```

③ 清理项目缓存，重新配置 CMake 并编译。

问题 2：重复声明成员函数，编译报错

- 问题描述：编译时报错：函数 “已经定义或声明成员函数”，无法通过编译。
- 错误原因：将同一个槽函数同时在 `private:` 和 `private slots:` 中各声明了一次；Qt 的 MOC 元对象编译器会为槽函数生成额外代码，重复声明导致符号冲突；
- 解决方法

① 打开头文件 `LSystem.h`；

删除普通成员区域 `private:` 中的重复函数声明；只保留 `private slots:` 下的唯一声明。

问题 3：背景音乐与生长音效无法播放

- 问题描述：代码运行无报错，但背景音乐和生长动画音效没有声音。
- 错误原因：资源路径错误：音频文件未正确加入 Qt 资源文件 `.qrc`，程序运行时找不到文件；播放器初始化顺序错误：未先设置 `QAudioOutput` 就直接播放；音量未设置：默认音量可能为 0 或过低；
- 解决方法

① 创建资源文件，将音频加入 `qrc`：

```
<qresource prefix="/audio">  
  
<file>audio/background.mp3</file></qresource>
```

② 按正确顺序初始化播放器：

```
m_audioOutput = new QAudioOutput(this);  
  
m_mediaPlayer->setAudioOutput(m_audioOutput);  
  
m_audioOutput->setVolume(0.3);  
  
m_mediaPlayer->setSource(QUrl("qrc:/audio/background.mp3"));
```

③ 播放前判断文件是否存在，避免无效路径。

问题 4：分形动画卡顿、绘制效率低

- 问题描述：迭代次数调高后，植物生长动画明显卡顿，界面响应变慢。

- 错误原因：字符串爆炸增长：L-System 迭代生成的指令字符串长度指数级上升，绘制压力大；每一帧都完整重算整条字符串，重复计算浪费性能；未限制最大迭代次数，导致程序计算量过载；
- 解决方法
 - ① 预先生成完整字符串，动画只做逐段绘制，不重复生成；
 - ② 限制迭代次数范围（1~7），避免过高迭代；
 - ③ 开启抗锯齿但不重复计算；
 - ④ 使用 Qt 双缓冲机制（Widget 默认开启）避免闪烁。

6. 实验收获

这次实验的收获不只是完成了一个分形植物程序，更重要的是把面向对象设计、图形绘制、音频播放和调试排错串成了一条完整链路。

- 理解了虚函数和虚析构函数的作用：基类接口通过重写实现多态，虚析构函数保证通过基类指针释放对象时资源不会泄漏。
- 掌握了 L-System 重写算法与海龟绘图的对对应关系，能够把字符串规则转化为图形指令。
- 学会了使用 QTimer 驱动动画，把“静态图形”改造成“生长过程”。
- 对 Qt Multimedia 的资源路径、qrc、临时文件导出和 FFmpeg 后端兼容性有了更具体的认识。
- 通过单元测试与控制台日志定位问题，认识到“先验证输入输出，再修复单点错误”比盲目改代码更高效。

遇到的问题主要集中在背景音乐无法播放和 qrc 资源路径不一致上。解决过程中，我先通过日志确认播放器已经创建，再进一步检查资源路径和资源层级，最后采用“资源别名 + 本地临时文件播放”的方式完成修复。这个过程让我更清楚地体会到：软件调试不是猜答案，而是用证据逐步缩小问题范围。

后续可以继续优化的方向包括：增加更多植物规则、增加自动季节切换、支持导出更高分辨率图片以及把用户偏好保存到配置文件中。

附录：

A. 关键代码位置

- 类定义与成员变量：L-System/LSystem.h
- 界面构造、重写规则、绘制逻辑：L-System/LSystem.cpp
- 资源配置：L-System/LSystem.qrc
- 阶段说明：第一阶段.md、第二阶段.md、第三阶段增强功能说明.md

B. AI 工具使用披露

在本次项目开发过程中，为提升开发效率与成果质量，合理使用了以下人工智能工具作为辅助：

1. 豆包（Doubao）

- 参考了项目的功能扩展与优化建议；
- 辅助完成部分模块功能说明文本的撰写与润色。

2. GPT（OpenAI ChatGPT）

- 协助进行用户界面设计与交互逻辑优化；
- 辅助代码实现与调试，包括 Bug 定位、语法修正与逻辑完善。

上述工具仅作为开发辅助手段，所有核心算法、架构设计与最终实现均由本人独立完成，报告中的分析、结论与代码实现均经过本人的独立验证与确认。